

به نام خدا

تکلیف اول پردازش تصویر

ماهان فلاح پور

۸۱۰۸۰۲۰۸۳



Q1:

$$R \ B \ G \longrightarrow \text{Grayscale}(i,j) = 0.2989 \times R(i,j) + 0.587 \times G(i,j) + 0.114 \times B(i,j)$$

10	10	16	28
9	6	26	37
15	25	13	22
32	15	87	39

10	10	16	28
9	6	26	37
15	25	13	22
32	15	87	39

$$\text{Grayscale} \longrightarrow \text{Binary}(i,j) = \begin{cases} 255 & \text{if Gray}(i,j) > \text{Threshold} \\ 0 & \text{otherwise} \end{cases}$$

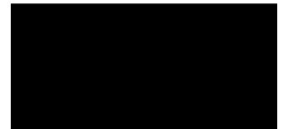
10	10	16	28
9	6	26	37
15	25	13	22
32	15	87	39

0	0	0	0
0	0	0	0
1	1	1	1
1	1	1	1

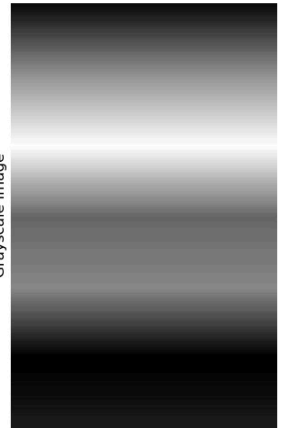


255 و 0

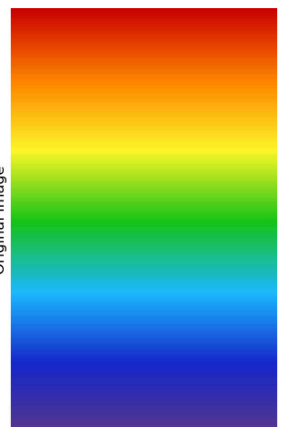
Binary Image



Grayscale Image



255 تا 0



پیکسل [0 تا 255] رنگها [0 تا 255]

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# بارگذاری تصویر با مسیر کامل
img = Image.open('/home/mahan/Documents/term6/Digital image processing/HW/HW1_image/Q1/download.jpeg')
img_array = np.array(img)

# تبدیل به خاکستری
R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
gray_img = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)

# تبدیل به باینری
threshold = 127
binary_img = np.where(gray_img > threshold, 255, 0).astype(np.uint8)

# نمایش تصاویر
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(img_array)
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(gray_img, cmap='gray')
axes[1].set_title('Grayscale Image')
axes[1].axis('off')

axes[2].imshow(binary_img, cmap='gray')
axes[2].set_title('Binary Image')
axes[2].axis('off')

plt.tight_layout()
plt.show()

# چاپ اطلاعات
print("Original image shape:", img_array.shape)
print("First 2x2 pixels (RGB):\n", img_array[:2, :2])

print("\nGrayscale image shape:", gray_img.shape)
print("First 2x2 pixels:\n", gray_img[:2, :2])

print("\nBinary image shape:", binary_img.shape)
print("First 2x2 pixels:\n", binary_img[:2, :2])
```

```
Original image shape: (177, 284, 3)
First 2x2 pixels (RGB):
[[[ 89  55 141]
  [ 89  55 141]]

 [[ 89  55 141]
  [ 89  55 141]]]

Grayscale image shape: (177, 284)
First 2x2 pixels:
[[74 74]
 [74 74]]

Binary image shape: (177, 284)
First 2x2 pixels:
[[[0 0]
  [0 0]]

 [[0 0]
  [0 0]]]
```

راه بد: هر دو پیکسل یکی رو برداریم ← بزگیات از دست یه تصویر دلتونه دار میشه

چه طور ی تصویر را
کوچک کنیم؟

Q2:

راه نو: پند پیکسل ا ترتیب نهم میانگینشون رو بگیریم

$$I_{new}(i,j) = \frac{1}{k^2} \sum_{x=0}^{k-1} \sum_{y=0}^{k-1} I(i \cdot k + x, j \cdot k + y)$$

مثال: $\begin{bmatrix} 100 & 120 & 140 & 160 \\ 110 & 130 & 150 & 170 \\ 200 & 220 & 240 & 225 \\ 210 & 230 & 250 & 255 \end{bmatrix}$

$$I_{new}(0,0) = \frac{100+120+110+130}{4} = 115$$

$$I_{new}(0,1) = \frac{140+160+150+170}{4} = 155$$

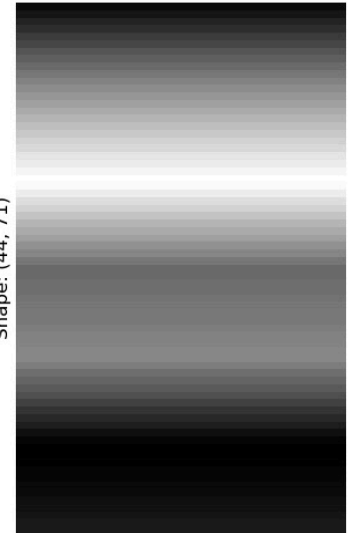
$$I_{new}(1,0) = \frac{200+220+210+230}{4} = 215$$

$$I_{new}(1,1) = \frac{240+255+250+255}{4} = 250$$

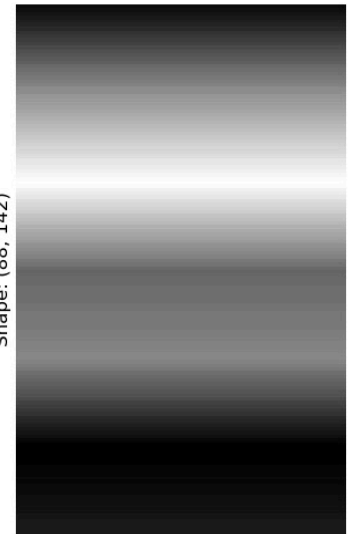


$\begin{bmatrix} 115 & 155 \\ 215 & 250 \end{bmatrix}$

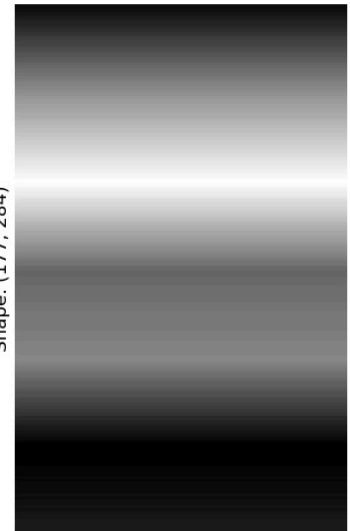
1/4 Scale (4x4 Avg)
Shape: (44, 71)



1/2 Scale (2x2 Avg)
Shape: (88, 142)



Original Grayscale
Shape: (177, 284)



```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# بارگذاری تصویر RGB
img = Image.open('/home/mahan/Documents/term6/Digital image processing/HW/HW1_image/Q1/download.jpeg')
img_array = np.array(img)

R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
gray_image = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)

def downsample_average(image, k):
    """
    Downsamples a 2D numpy array by averaging over k x k blocks.
    """
    H, W = image.shape

    # تبدیل شود (برای جلوگیری از خطای ابعاد) K بریدن ابعاد تصویر تا به مضرب از
    H_new = (H // k) * k
    W_new = (W // k) * k
    img_cropped = image[:H_new, :W_new]

    # محاسبه میانگین روی محورهای مربوط به بلوکها K*K تغییر شکل آرایه به بلوکهای
    downsampled_img = img_cropped.reshape(H_new // k, k, W_new // k, k).mean(axis=(1, 3))

    return downsampled_img.astype(np.uint8)

# 1. 1/2 بلوکهای (2) کوچک کردن به اندازه
img_half_scale = downsample_average(gray_image, 2)

# 2. 1/4 بلوکهای (4) کوچک کردن به اندازه
img_quarter_scale = downsample_average(gray_image, 4)

# چاپ ابعاد برای مقایسه
print(f"Original Grayscale Shape: {gray_image.shape}")
print(f"1/2 Scale Image Shape: {img_half_scale.shape}")
print(f"1/4 Scale Image Shape: {img_quarter_scale.shape}")

# نمایش تصاویر
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(gray_image, cmap='gray')
axes[0].set_title('Original Grayscale\nShape: {}'.format(gray_image.shape))
axes[0].axis('off')

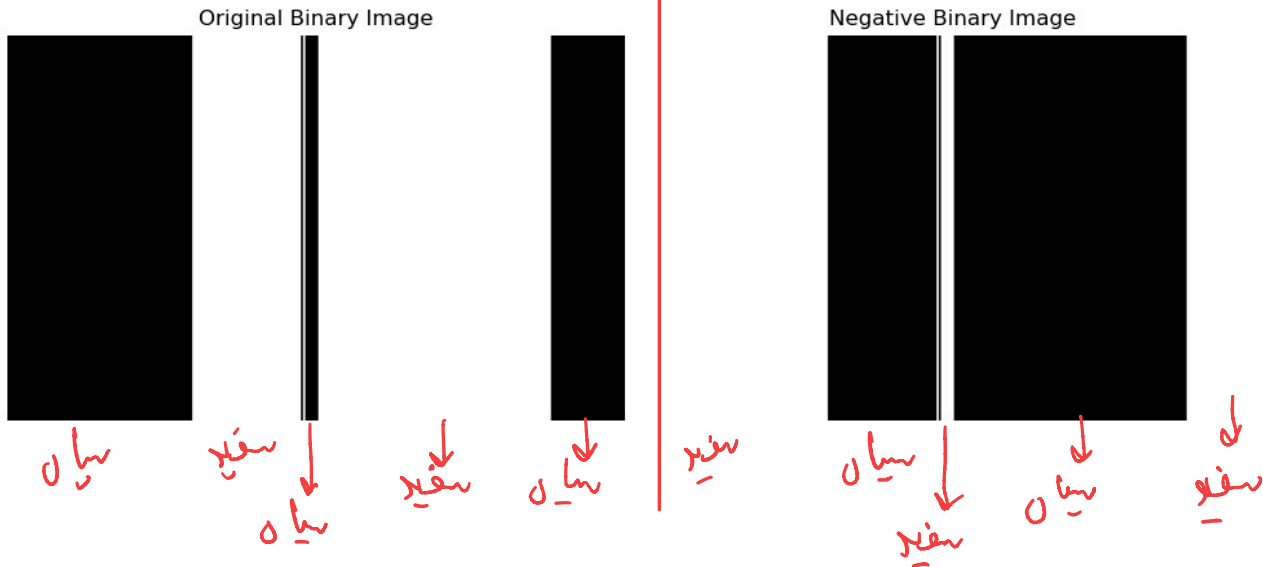
axes[1].imshow(img_half_scale, cmap='gray')
axes[1].set_title('1/2 Scale (2x2 Avg)\nShape: {}'.format(img_half_scale.shape))
axes[1].axis('off')

axes[2].imshow(img_quarter_scale, cmap='gray')
axes[2].set_title('1/4 Scale (4x4 Avg)\nShape: {}'.format(img_quarter_scale.shape))
axes[2].axis('off')

plt.tight_layout()
plt.show()
```

Original Grayscale Shape: (177, 284)
1/2 Scale Image Shape: (88, 142)
1/4 Scale Image Shape: (44, 71)

Q3:



```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

img = Image.open('/home/mahan/Documents/term6/Digital image processing/HW/Hw1_image/Q1/download.jpeg')
img_array = np.array(img)

R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
gray_array = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)

# تبدیل به بااینری با آستانه 128 (اعدادی کمتر از 128 صفر و بقیه 255 میشوند)
threshold = 128
binary_image = np.where(gray_array < threshold, 0, 255).astype(np.uint8)

# ایجاد تصویر نگاتیو به صورت دستی با فرمول ریاضی 2.
# کم کردن تک تک مقادیر از 255
negative_image = 255 - binary_image

# از گوشه تصویر را جاب میکنیم (فقط یک بخش 5) جاب و مقایسه مقادیر آرایه ها 4.
print("--- (بالا سمت چپ 5x5 بخش 5) مقایسه مقادیر آرایه ---")
print("Original Binary Array:")
print(binary_image[0:5, 0:5])
print("\nNegative Binary Array:")
print(negative_image[0:5, 0:5])

# نمایش هر دو تصویر 3.
fig, axes = plt.subplots(1, 2, figsize=(10, 5))

axes[0].imshow(binary_image, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Original Binary Image')
axes[0].axis('off')

axes[1].imshow(negative_image, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Negative Binary Image')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

(آستانه 128 پیپ 5x5 بخش 5)

Original Binary Array:

```
[[0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]
 [0 0 0 0 0]]
```

Negative Binary Array:

```
[[255 255 255 255 255]
 [255 255 255 255 255]
 [255 255 255 255 255]
 [255 255 255 255 255]
 [255 255 255 255 255]]
```


تصحیح گاما: یک عملیات غیر خطی که برای تنظیم روشنایی تصویر استفاده می‌شود، دلیل اصلی استفاده از آن اینست که چشم انسان در مانیتورها نور را به صورت فکلی درک نمی‌کنند، چشم انسان نسبت به تغییرات در سایه‌ها بسیار حساس‌تر است از تغییرات در نورانی روشن است. تصحیح گاما به ما اجازه می‌دهد تا این رفتار غیر فکلی رو مبدان کنیم و در کل با این روش ویژگیات پنهان در نورانی تاریک یا بسیار روشن آشکار شوند.

Q4:

پیکسل ورودی در بازه [0, 255] نرمالسازی می‌شوند

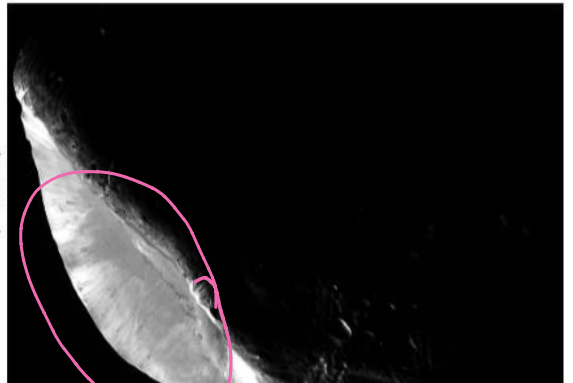
$$I_{out} = 255 \times \left(\frac{I_{in}}{255} \right)^\gamma$$

بعد به توان γ می‌رسد

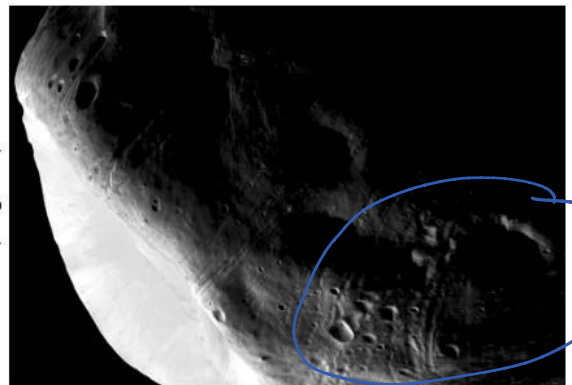
آنر سر برمی‌گردد دوباره به بازه [0, 255]

برای $\gamma < 1$ → فرنیای تاریک معلوم می‌شود
 برای $\gamma > 1$ → فرنیای روشن معلوم می‌شود
 برای $\gamma = 1$ → تغییری ایجاد نمی‌شود

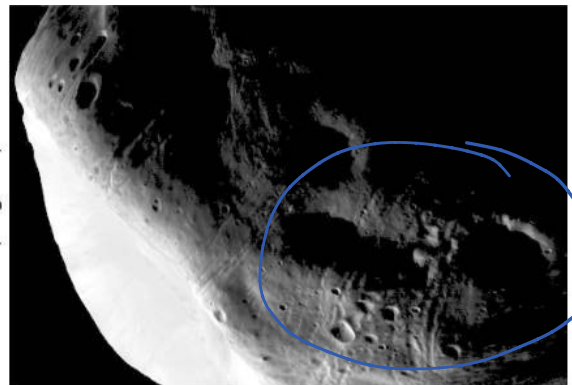
Gamma Correction ($\gamma = 2.0$)
(Darker)



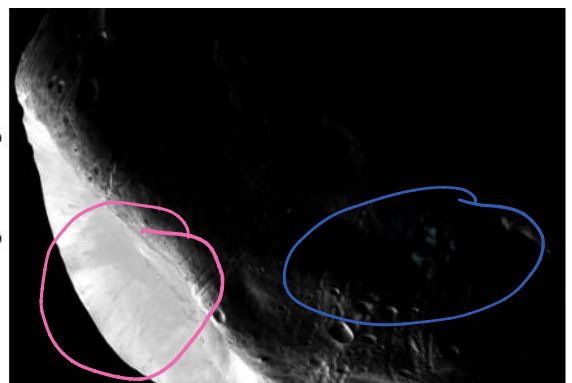
Gamma Correction ($\gamma = 0.5$)
(Brighter)



Gamma Correction ($\gamma = 0.3$)
(Brighter)



Original Image



```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 1. بارگذاری تصویر
img = Image.open('/home/mahan/Documents/term6/Digital image processing/HW/HW1_image/Q4/question4.tif')
img_array = np.array(img)

# بررسی ابعاد تصویر
if img_array.ndim == 3: # تصویر RGB
    R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
    gray_array = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)
elif img_array.ndim == 2: # تصویر از قبل grayscale
    gray_array = img_array
else:
    raise ValueError("فرمت تصویر پشتیبانی نشده")

# 2. تابع اعمال تصحیح گاما
def apply_gamma_correction(image_array, gamma):
    normalized_img = image_array / 255.0
    corrected_img = np.power(normalized_img, gamma)
    final_img = (corrected_img * 255).astype(np.uint8)
    return final_img

# 3. اعمال تصحیح برای دو مقدار گاما
gamma_1 = 0.5
gamma_2 = 2.0

img_gamma_05 = apply_gamma_correction(gray_array, gamma_1)
img_gamma_20 = apply_gamma_correction(gray_array, gamma_2)

# 4. نمایش نتایج (SyntaxWarning را raw string)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(gray_array, cmap='gray', vmin=0, vmax=255)
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(img_gamma_05, cmap='gray', vmin=0, vmax=255)
axes[1].set_title(fr'Gamma Correction (\gamma = {gamma_1})' + '\n(Brighter)')
axes[1].axis('off')

axes[2].imshow(img_gamma_20, cmap='gray', vmin=0, vmax=255)
axes[2].set_title(fr'Gamma Correction (\gamma = {gamma_2})' + '\n(Darker)')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```

هدف اصلی فیلترهای پایین‌گذر کاهش نویز و محو کردن تصویر است، همون طوره و دومین تغییر ناگهانی شد روشایی به عنوان رنگش بالا شفافته‌ی شونده. فیلتر هموار ساز با گرفتن میانگین از یکدسل های مجاور، این تغییر ناگهانی را صاف کرده و نقطه بردن کلی تر (رنگش پایین) ا بازه عبوری دهو

Q5:

H روی تصویر درکت داده میشه مرکزش روی هر پیکسل از تصویر قرار میگیره ، مقدار یکدسل دیو برابر خواهد بود با مجموع مقادیر پیکسل مرکزی و 8 همسایه اطراف آن مرتب در مرتب و

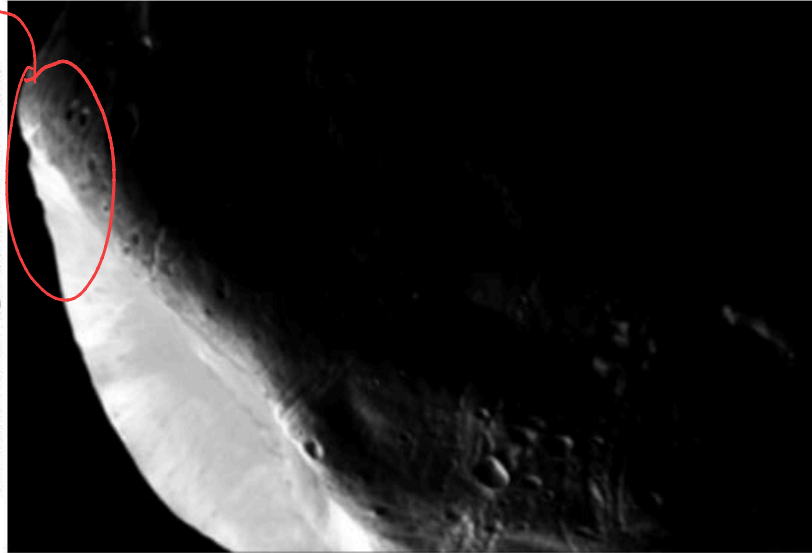
$$H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$\frac{1}{9}$	1	1	1
	1	1	1
	1	1	1

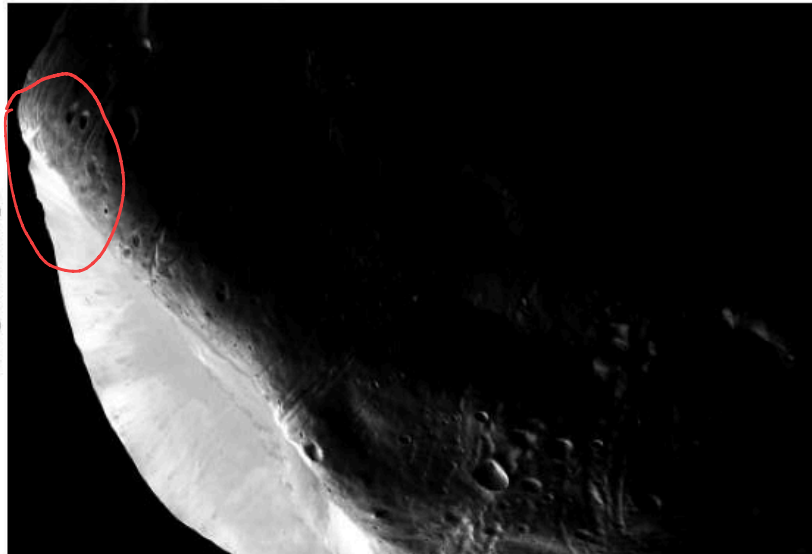
Mask or
Window or
Template

از تارپ بودن در آمده

Smoothed Image (3x3 Mean Filter)



Original Image



```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# 1. Load the image and convert to grayscale
img = Image.open('/home/mahan/Documents/term6/Digital image processing/HM/HM1_image/Q4/question4.tif')
img_array = np.array(img, dtype=float)

if img_array.ndim == 3: # تصویر RGB
    R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
    gray_array = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)
elif img_array.ndim == 2: # تصویر از قبل grayscale
    gray_array = img_array
else:
    raise ValueError("فرمت تصویر پشتیبانی نشده")

# 2. Zero-padding
# در هر طرف داریم (padding) نیاز به 1 پیکسل حاشیه 3x3 برای گرید
pad_width = 1
padded_img = np.pad(img_array, pad_width=pad_width, mode='constant', constant_values=0)

# آرایه خالی برای تصویر خروجی
rows, cols = img_array.shape
smoothed_img = np.zeros((rows, cols), dtype=float)

# 3. Manual Convolution using nested loops
# حرکت روی تمام پیکسلهای تصویر انجام
for i in range(rows):
    for j in range(cols):
        # اطراف پیکسل (از روی تصویر بد شده) 3x3 استخراج منطقه
        region = padded_img[i : i+3, j : j+3]
        # محاسبه میانگین مقادیر این منطقه (معادل اعمال مانت میانگینگیر)
        smoothed_img[i, j] = np.sum(region) / 9.0

# 4. Convert back to uint8 for proper display
smoothed_img_uint8 = np.clip(smoothed_img, 0, 255).astype(np.uint8)

# نمایش تصاویر
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.imshow(img_array, cmap='gray', vmin=0, vmax=255)
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(smoothed_img_uint8, cmap='gray', vmin=0, vmax=255)
plt.title('Smoothed Image (3x3 Mean Filter)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

ایده شارپ کردن با افزودن لاپلاسین و لاپلاسین

مشتق دوم تصویر را محاسبه می کنند

در نوا می flat

نظری فیلتر لاپلاسین تقریباً همزه چون تقریباً شدت روشنی صفر در لبه ها // // // در یک سمت لبه مقدار مثبت بزرگ و در سمت دیگر مقدار منفی بزرگ می گیرد (نشان دهنده تغییر ناگهانی)

Q6:

$$\nabla^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

0	1	0
1	-4	1
0	1	0

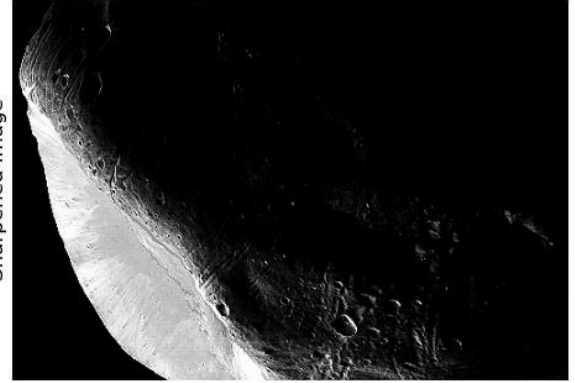
The center of the mask is negative

ل = -1 ← اگر مرکز ماسک - ل = +1 ⇒ مرکز ماسک +

$$I_{\text{sharpened}}(x, y) = I_{\text{original}}(x, y) + c \nabla^2 I(x, y)$$

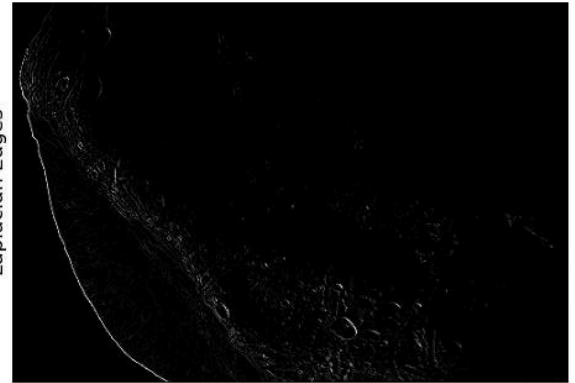
برای شارپ کردن این لبه های استخراج شده را با یک مقیاس به تصویر اصلی اضافه می کنیم
این کار باعث میشه کنتراست در لبه افزایش یابد و تصاویر نهایی واضح تر و شارپ تر به نظر برسد

$$P_{hb}(x, y) = \begin{cases} kP(x, y) - \nabla^2 P(x, y) & \longrightarrow \text{The center of the mask is negative} \\ kP(x, y) + \nabla^2 P(x, y) & \longrightarrow \text{The center of the mask is positive} \end{cases}$$



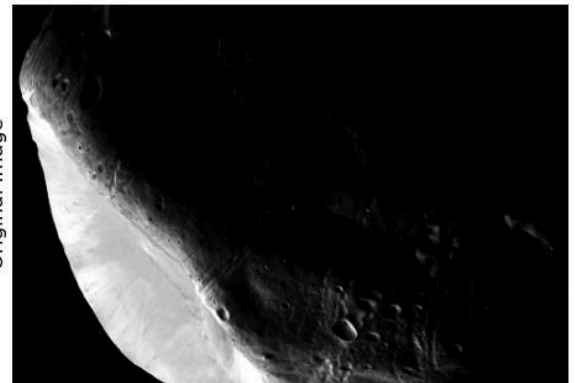
Sharpened Image

||



Laplacian Edges

+



Original Image

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

def manual_laplacian_sharpening(image_path, scale=1.0):
    # Grayscale بارگذاری تصویر و تبدیل به
    img = Image.open(image_path)
    img_array = np.array(img, dtype=np.float32)

    if img_array.ndim == 3: # تصویر RGB
        R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
        gray_array = (0.2989 * R + 0.5876 * G + 0.1140 * B).astype(np.uint8)
    elif img_array.ndim == 2: # تصویر از قبل grayscale
        gray_array = img_array
    else:
        raise ValueError("فرمت تصویر پشتیبانی نمیگردد")

    # ابعاد تصویر
    rows, cols = img_array.shape

    # تعریف ماسک لاپلاسین (هرگز مثبت برای جمع صحتی)
    laplacian_mask = np.array([[ 0, -1, 0],
                               [-1, 4, -1],
                               [ 0, -1, 0]], dtype=np.float32)

    # ایجاد آرایه خالی برای خروجی لبه ها
    edges = np.zeros_like(img_array)

    # اعمال فیلتر به صورت دستی
    for i in range(1, rows - 1):
        for j in range(1, cols - 1):
            region = img_array[i-1:i+2, j-1:j+2]
            edges[i, j] = np.sum(region * laplacian_mask)

    # جمع کردن لاپلاسین با تصویر اصلی
    sharpened_array = img_array + (scale * edges)

    # محدود کردن مقادیر بین 0 و 255
    sharpened_array = np.clip(sharpened_array, 0, 255).astype(np.uint8)
    edges_clipped = np.clip(edges, 0, 255).astype(np.uint8)

    return img_array.astype(np.uint8), edges_clipped, sharpened_array

# اجرای تابع
img_orig, img_edges, img_sharp = manual_laplacian_sharpening(
    '/home/maham/Documents/term6/Digital image processing/HM/HM1_image/Q4/question4.tif',
    scale=1.0
)

# نمایش تصاویر
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(img_orig, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(img_edges, cmap='gray')
axes[1].set_title('Laplacian Edges')
axes[1].axis('off')

axes[2].imshow(img_sharp, cmap='gray')
axes[2].set_title('Sharpened Image')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```

به تقریباً بیشترش سیاه است

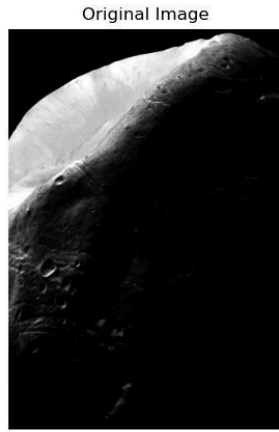
Q7:

$$S_k = T(r_k) = \sum_{j=0}^k p_r(r_j)$$

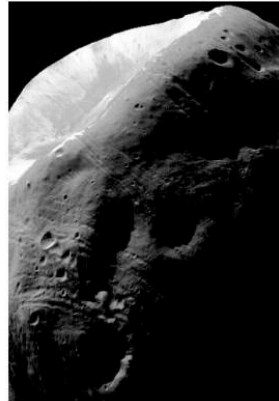
تعداد کل پیکسل‌ها

تعداد پیکسل‌ها

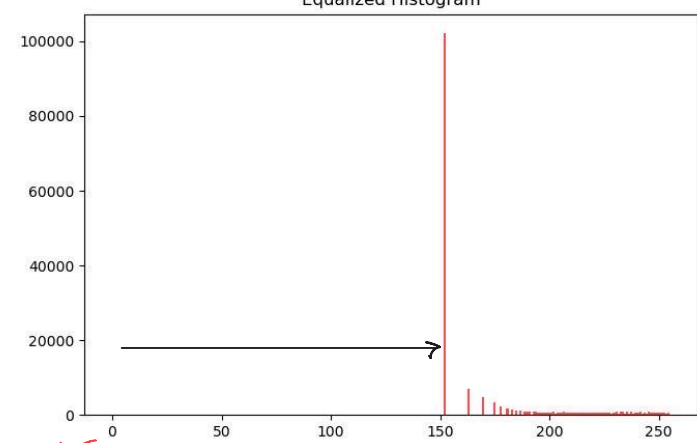
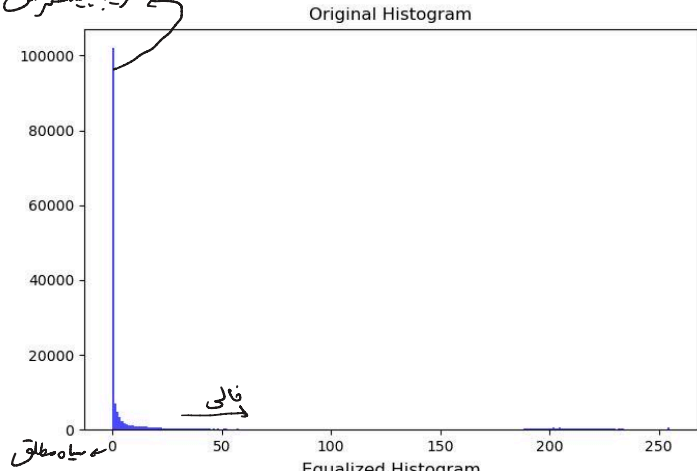
تعداد کل پیکسل‌ها



Original Image



Equalized Image



$$p(r_k) = \frac{n_k}{N}$$

تعداد پیکسل‌های روشنایی k ام

سطح روشنایی k ام

تعداد کل پیکسل‌های تصویر

$$S_k = T(r_k) = (L-1) \sum_{j=0}^k p(r_j)$$

تعداد کل سطح روشنایی ممکن

باین روشن مقدار CDF را به بازه [0-1] نگاشت می‌دهیم

معادله سازی هیستوگرام با بخش یون متادیر روشنایی پیکسل‌ها در کل بازه موجود که فرکانس را افزایش می‌دهد، در تعدادی که کمتر است باینی دارند هیستوگرام در یک محدوده یونیک متکلم شدن.

با استفاده از تابع (FD) خواص بین سطح روشنایی نه فرکانس باین دارند کشیده و بازی شوند، در نتیجه، تفاوت‌های طرف در روشنایی برجسته تر شوند و باینی که در نوای تاریک روشن پنهان بود در تصویر نهایی قابل مشاهده می‌شوند.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image = Image.open('/home/mahan/Documents/term6/Digital image processing/HW/HW1_image/Q4/question4.tif')
img_array = np.array(image)

if img_array.ndim == 3:
    R, G, B = img_array[:, :, 0], img_array[:, :, 1], img_array[:, :, 2]
    gray_array = (0.2989 * R + 0.5870 * G + 0.1140 * B).astype(np.uint8)
elif img_array.ndim == 2:
    gray_array = img_array
else:
    raise ValueError("فرمت تصویر پشتیبانی نمی‌شود")

L = 256

hist, bins = np.histogram(img_array.flatten(), bins=L, range=[0, 255])
pdf = hist / img_array.size

cdf = np.cumsum(pdf)

transformation_map = np.round(cdf * (L - 1)).astype(np.uint8)

equalized_img_array = transformation_map[img_array]

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

axes[0, 0].imshow(img_array, cmap='gray')
axes[0, 0].set_title("Original Image")
axes[0, 0].axis('off')

axes[0, 1].hist(img_array.flatten(), bins=L, range=[0, 255], color='blue', alpha=0.7)
axes[0, 1].set_title("Original Histogram")

axes[1, 0].imshow(equalized_img_array, cmap='gray')
axes[1, 0].set_title("Equalized Image")
axes[1, 0].axis('off')

axes[1, 1].hist(equalized_img_array.flatten(), bins=L, range=[0, 255], color='red', alpha=0.7)
axes[1, 1].set_title("Equalized Histogram")

plt.tight_layout()
plt.show()
```